

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы №2
по дисциплине «Функциональное программирование»
Вариант №17.

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Моторин Д.Е.

«_____» _____ 2025г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Постановка задачи	4
2 Разработка модели финансовых транзакций	5
2.1 Определение типов данных	5
2.2 Функции	5
3 Проверка корректности	7
Выводы	9
Список использованных источников	10
Приложение 1 «Исходный код программы»	11

Введение

Финансовая транзакция - это запись о движении денежных средств между сторонами в результате любой сделки или операции с деньгами или финансовыми активами.

В каждой транзакции должны быть: уникальный идентификатор, сумма, тип операции и ее описание. Нередко добавляют точную дату и время, валюту, источника и получателя денежных средств.

Основные типы операций связаны с направлением денежных потоков:

1. Доход - поступление денег на счет.
2. Расход - убытие средств со счета.
3. Перевод - перемещение денег между своими счетами.
4. Депозит - накопление средств.

Транзакции подчиняются некоторым бизнес-правилам:

1. Данные должны быть целостны: сумма всегда положительная, все обязательные поля заполнены.
2. Данные должны быть валидны: счета, валюты, даты, персональная информация корректная и подтвержденная.
3. Завершенные транзакции не могут быть изменены.
4. Во время выполнения транзакции параллельно ей не может выполняться другая транзакция, использующая те же счета. В случае сбоя транзакции движение всех денежных средств отменяется.
5. Каждой транзакции присваивается только одна категория.

Модели финансовых транзакций используются для учета денежных потоков, налоговой отчетности, анализа и планирования бюджета.

1 Постановка задачи

Цель: разработка модели финансовых транзакций, используя пользовательские типы данных и рекурсивные функции.

Задачи:

1. Определить тип транзакции `TransactionType`.
2. Определить тип транзакции `Transaction`.
3. Написать функцию `excessStep`.
4. Написать рекурсивную функцию `classifyFunc`.

2 Разработка модели финансовых транзакций

2.1 Определение типов данных

Тип транзакции `TransactionType` был определен как тип-сумма, который представляет возможные категории финансовых операций: доход, расход, перевод и депозит. Тип получает возможность сравнения и вывода на печать.

Листинг 1. `TransactionType`

```
1 data TransactionType = Income | Expense | Transfer | Deposit
2 deriving (Eq, Show)
```

Тип `Transaction` был определен как тип-произведение, который объединяет несколько полей в одну структуру: идентификатор транзакции типа `Int`, сумму типа `Double`, тип транзакции типа `TransactionType`, описание транзакции типа `String`. Тип получает возможность сравнения и вывода на печать.

Листинг 2. `TransactionType`

```
1 data Transaction = Transaction {
2   transactionId :: Int
3   , summa :: Double
4   , transactionType :: TransactionType
5   , descr :: String
6 } deriving (Eq, Show)
```

2.2 Функции

Функция `excessStep` сравнивает сумму транзакции с порогом. Она принимает на вход пороговое значение типа `Double` и транзакцию типа `Transaction`. Если сумма транзакции превышает порог, функция возвращает `True`, иначе `False` - значения типа `Bool`.

Листинг 3. `excessStep`

```
1 excessStep :: Double -> Transaction -> Bool
2 excessStep step transaction = summa transaction > step
```

Функция `classifyFunc` классифицирует список транзакций по их принадлежности к двум предикатам, разбивая на четыре группы: удовлетворяет только первому предикату (`onlyP`), удовлетворяет только второму предикату (`onlyQ`), удовлетворяет обоим предикатам (`both`), не удовлетворяет ни одному предикату (`neither`). Функция принимает на вход два

предиката и список транзакций типа `Transaction`, возвращает кортеж из четырех списков.

Листинг 4. `classifyFunc`

```
1  classifyFunc :: (a -> Bool) -> (a -> Bool) -> [a] -> ([a],[a],[a],[a])
2  classifyFunc _ _ [] = ([],[],[a],[a])
3  classifyFunc p q (x : xs) =
4  let (onlyP, onlyQ, both, neither) = classifyFunc p q xs
5  in case (p x, q x) of
6  (True, True) -> (onlyP, onlyQ, x:both, neither)
7  (True, False) -> (x:onlyP, onlyQ, both, neither)
8  (False, True) -> (onlyP, x:onlyQ, both, neither)
9  (False, False) -> (onlyP, onlyQ, both, x:neither)
```

Если список транзакций пустой - в результате получается кортеж из четырех пустых списков. Иначе список разделяется на хвост и голову, функция рекурсивно применяется для хвоста. Для текущего элемента («головой») проверяется выполнение предикатов, по результатам элемент записывается в соответствующий список.

3 Проверка корректности

Для тестирования корректности функций была создана транзакция myTransaction с суммой 200.5.

Результаты работы функции excessStep с различными пороговыми значениями представлены на рисунке 1.

```
ghci> excessStep 5.0 myTransaction
True
ghci> excessStep 1005.0 myTransaction
False
```

Рис. 1. Проверка корректности функции excessStep

Для проверки корректности второй функции было создано два предиката: isIncome на соответствие типа операции типу Income и isSmall на фильтр по сумме не более 80.0. Также было создано два списка транзакций: один заполненный значениями, второй пустой. Пример работы функции classifyFunc с пустым списком представлен на рисунке 2.

```
ghci> onlyP
[]
```

Рис. 2. Проверка с пустым списком

Отображение непустого списка представлено на рисунке 3. На рисунках 4 - 7 представлены результаты классификации компонентов списка по их принадлежности к двум предикатам.

```
(([Transaction {transactionId = 1, summa = 200.5, transactionType = Income, descr = "Grant"},
Transaction {transactionId = 3, summa = 500.0, transactionType = Income, descr = "Salary"}],
[Transaction {transactionId = 2, summa = 50.0, transactionType = Expense, descr = "Dormitory"},
Transaction {transactionId = 5, summa = 20.0, transactionType = Deposit, descr = "Percent"},
Transaction {transactionId = 6, summa = 30.0, transactionType = Income, descr = "Present"},
Transaction {transactionId = 4, summa = 120.0, transactionType = Transfer, descr = "Birthday"}])
```

Рис. 3. Вывод непустого списка

```
ghci> onlyP
[Transaction {transactionId = 1, summa = 200.5, transactionType = Income, descr = "Grant"},
Transaction {transactionId = 3, summa = 500.0, transactionType = Income, descr = "Salary"}]
```

Рис. 4. Только первый предикат

```
ghci> onlyQ
[Transaction {transactionId = 2, summa = 50.0, transactionType = Expense, descr = "Dormitory"},
Transaction {transactionId = 5, summa = 20.0, transactionType = Deposit, descr = "Percent"}]
```

Рис. 5. Только второй предикат

```
ghci> both
[Transaction {transactionId = 6, summa = 30.0, transactionType = Income, descr = "Present"}]
```

Рис. 6. Оба предиката

```
ghci> neither
[Transaction {transactionId = 4, summa = 120.0, transactionType = Transfer, descr = "Birthday"}]
```

Рис. 7. Ни один предикат

По результатам тестирования функции видно, что все элементы распределяются по нужным категориям в соответствии с заданными предикатами.

Выводы

В ходе работы была успешно реализована модель финансовых транзакций. Для этого были определены тип транзакции `TransactionType` и тип `Transaction`, объединяющий в себе несколько полей. Были реализованы функция `excessStep` для сравнения суммы транзакции с пороговым значением, а также рекурсивная функция `classifyFunc`, классифицирующая транзакцию в зависимости от ее принадлежности к предикатам. Тестирование программы показало, что все функции работают корректно и обрабатывают частные случаи.

Список литературы

- [1] Курт У. «Программируй на Haskell». М.: Издательство, 2019. 320 стр.
- [2] Липовача М. «Изучай Haskell во имя добра». М.: ДМК Пресс, 2012. 490 стр.

Приложение 1 «Исходный код программы»

Листинг 5. Код работы

```
1 -- Задание 2, вариант 17. Разработать модель финансовых транзакций
2
3 -- Определить типы данных: тип транзакции TransactionType как типсумму-
  например(: Income, Expense, Transfer)
4 data TransactionType = Income | Expense | Transfer | Deposit
5 deriving (Eq, Show)
6 -- Transaction как типпроизведение- с полями: идентификатор транзакции, сумма,
  тип транзакции, описание транзакции
7 data Transaction = Transaction {
8   transactionId :: Int
9   , summa :: Double
10  , transactionType :: TransactionType
11  , descr :: String
12 } deriving (Eq, Show)
13
14 myTransaction :: Transaction
15 myTransaction = Transaction {
16   transactionId = 1
17   , summa = 200.5
18   , transactionType = Income
19   , descr = "Grant"
20 }
21 -- Написать функцию, которая принимает пороговое значение и транзакцию, возвращает
  True, если сумма транзакции превышает порог
22 excessStep :: Double -> Transaction -> Bool
23 excessStep step transaction = summa transaction > step
24 -- excessStep 5.0 myTransaction
25 -- excessStep 1005.0 myTransaction
26
27
28 -- Написать рекурсивную функцию, которая принимает два предиката и список
  транзакций, возвращает кортеж из четырех списков:
29 -- удовлетворяет только первому предикату, удовлетворяет только второму предикату,
  удовлетворяет обоим предикатам, не удовлетворяет ни одному предикату
30 -- Пример типа: (a -> Bool) -> (a -> Bool) -> [a] -> ([a],[a],[a],[a])
31 -- Пример использования: фильтр по типу операции и фильтр по сумме транзакции
32 classifyFunc :: (a -> Bool) -> (a -> Bool) -> [a] -> ([a],[a],[a],[a])
33 classifyFunc _ _ [] = ([],[],[],[ ])
34 classifyFunc p q (x : xs) =
35 let (onlyP, onlyQ, both, neither) = classifyFunc p q xs
36 in case (p x, q x) of
37 (True, True) -> (onlyP, onlyQ, x:both, neither)
38 (True, False) -> (x:onlyP, onlyQ, both, neither)
39 (False, True) -> (onlyP, x:onlyQ, both, neither)
40 (False, False) -> (onlyP, onlyQ, both, x:neither)
41
42
43 transactionsBox :: [Transaction]
44 transactionsBox = [
45   Transaction 1 200.5 Income "Grant"
46 , Transaction 2 50.0 Expense "Dormitory"
47 , Transaction 3 500.0 Income "Salary"
```

```

48 , Transaction 4 120.0 Transfer "Birthday"
49 , Transaction 5  20.0 Deposit  "Percent"
50 , Transaction 6  30.0 Income   "Present"
51 ]
52
53 tb2 :: [Transaction]
54 tb2 = []
55
56 isIncome :: Transaction -> Bool
57 isIncome t = transactionType t == Income
58
59 isSmall :: Transaction -> Bool
60 isSmall t = summa t < 80.0
61
62 example :: ([Transaction],[Transaction],[Transaction],[Transaction])
63 example = classifyFunc isIncome isSmall transactionsBox
64
65 example2 :: ([Transaction],[Transaction],[Transaction],[Transaction])
66 example2 = classifyFunc isIncome isSmall tb2
67
68 -- let (onlyP, onlyQ, both, neither) = example

```